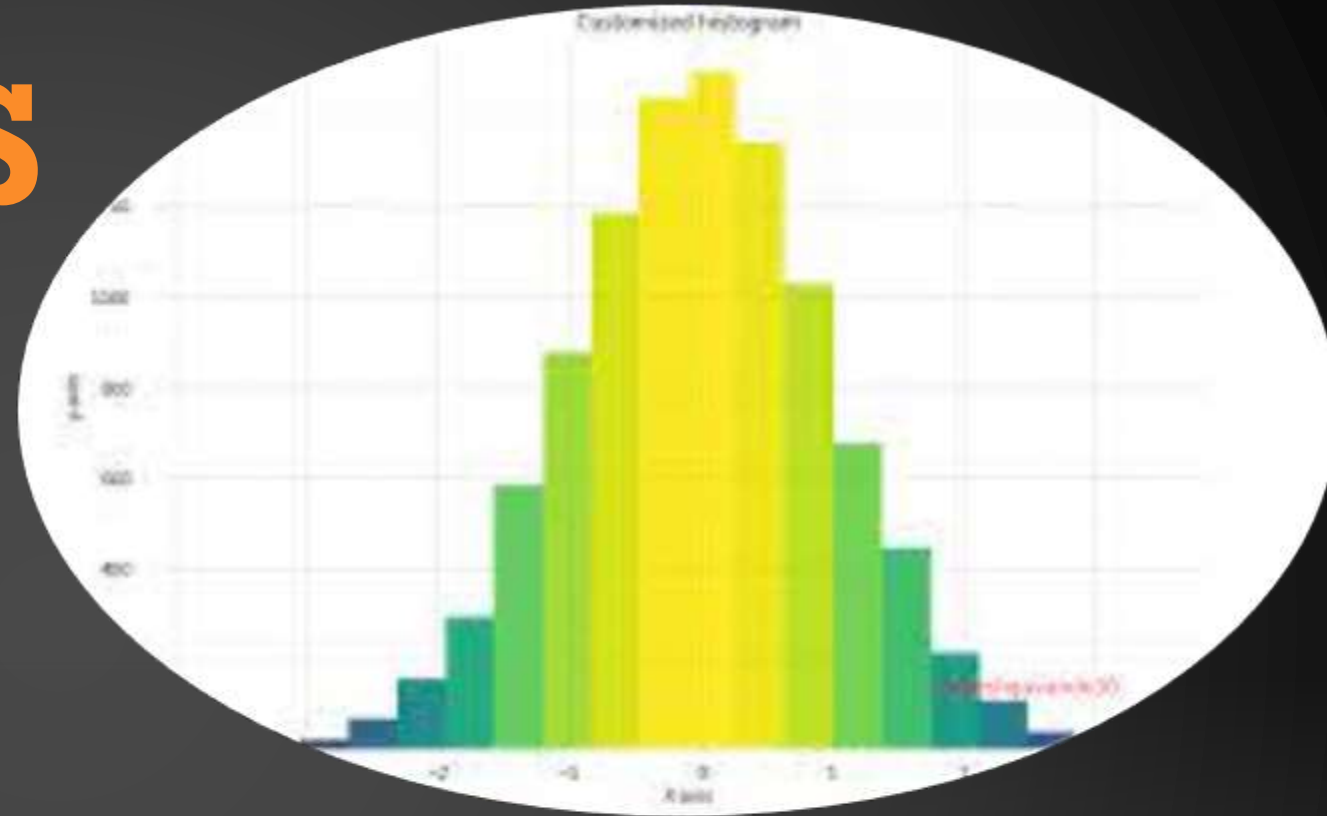


HISTOGRAMS

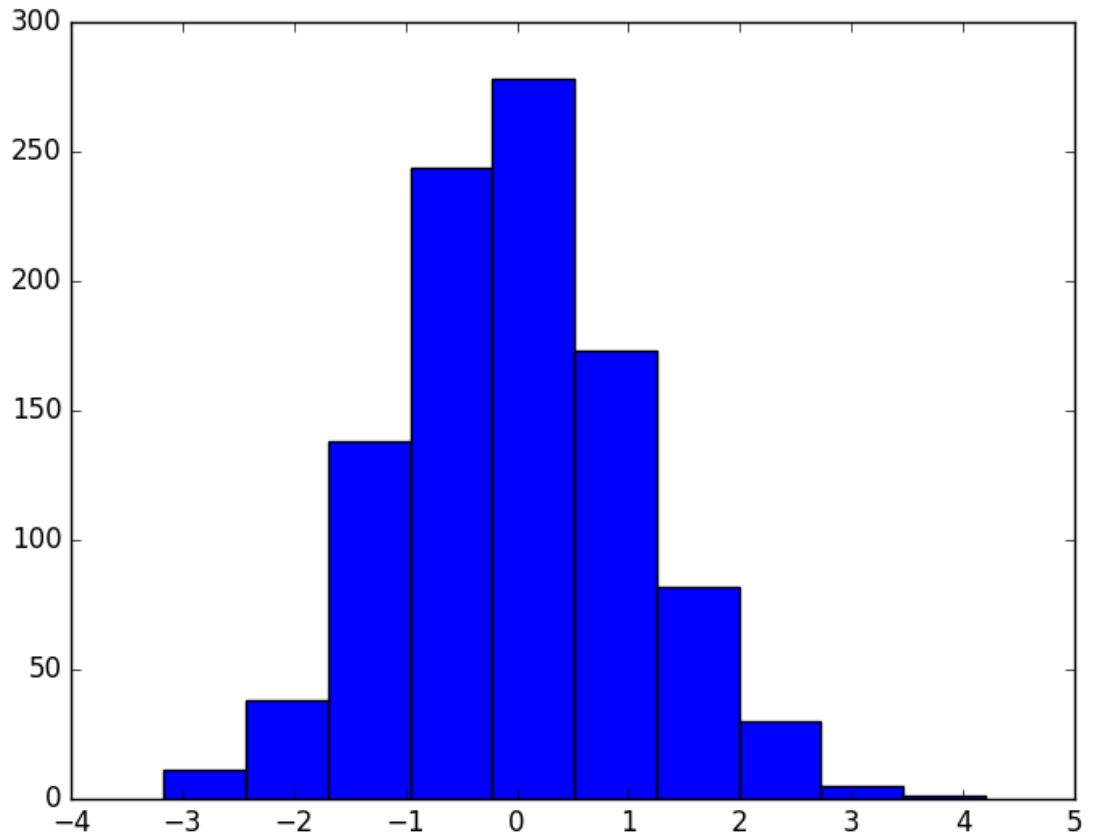
- **PLT.HIST()**
- **NP.HISTOGRAM()**
- **PLT.HIST2D()**
- **NP.HISTOGRAM2D()**
- **PLT.HEXBIN()**



HISTOGRAMS

- A simple histogram can be a great first step in understanding a dataset.
- Using `plt.hist()` function – to create a 1-d histogram for the given data

```
import matplotlib.pyplot as plt
import numpy as np
plt.style.use('classic')
data = np.random.randn(1000)
plt.hist(data)
plt.show()
```



HISTOGRAMS

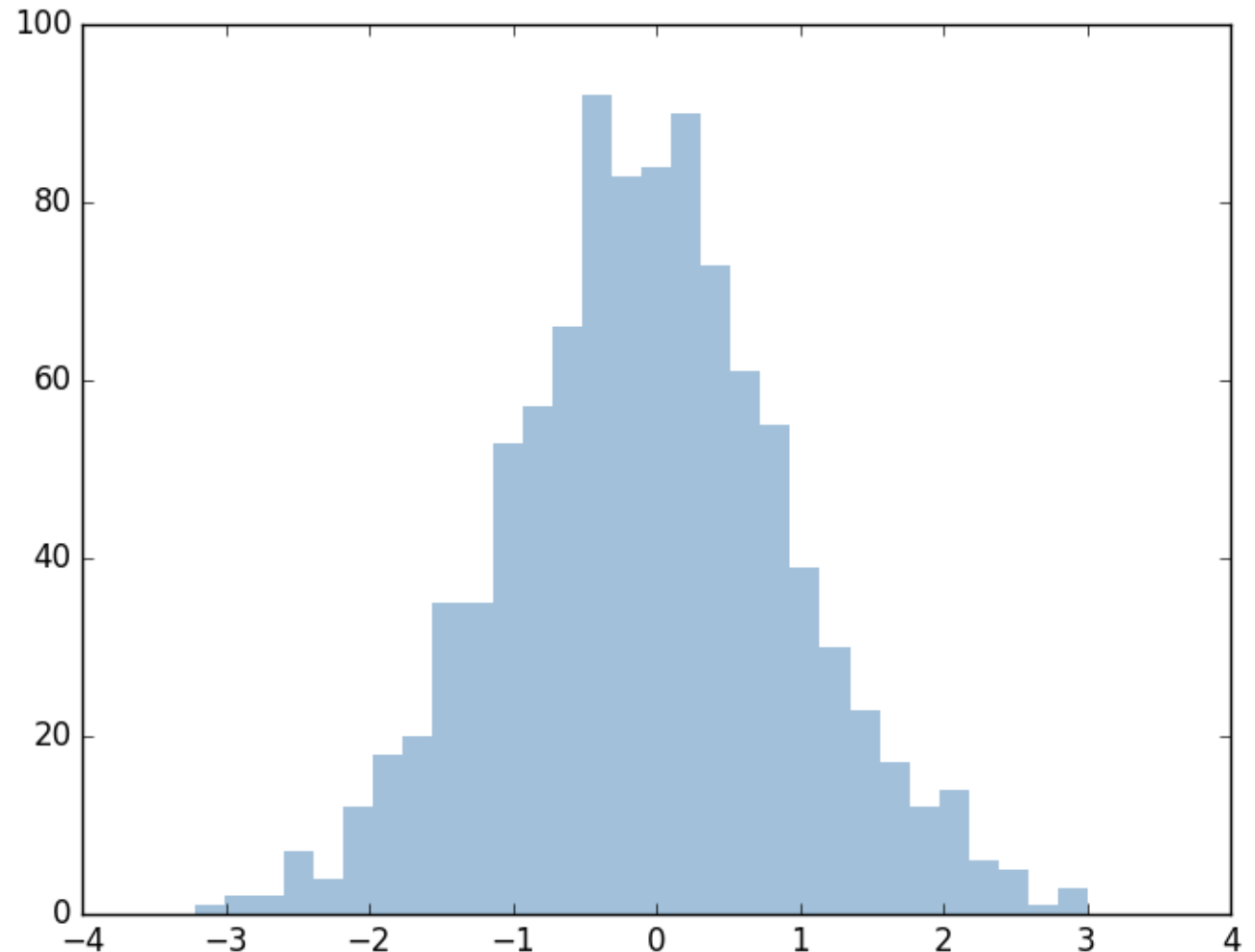
- The hist() function has many options to tune both the calculation and the display

```
import matplotlib.pyplot as plt
import numpy as np
plt.style.use('classic')
data = np.random.randn(1000)
plt.hist(data,      bins=30,      normed=True,      alpha=0.5,
histtype='stepfilled', color='steelblue', edgecolor='none');
plt.show()
```

HISTOGRAMS

- The `hist()` function has many options to tune both the calculation and the display

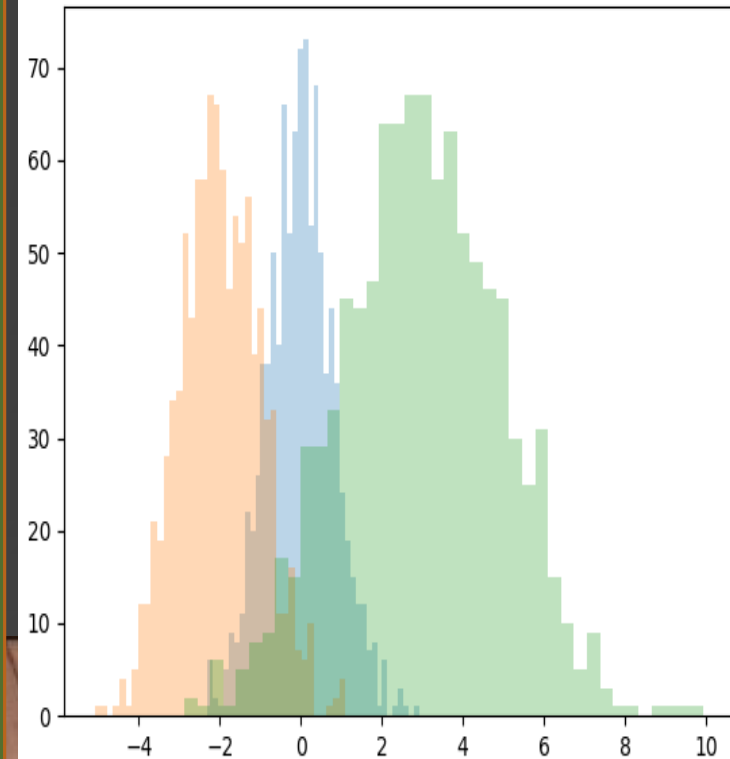
```
import matplotlib.pyplot as plt
import numpy as np
plt.style.use('classic')
data = np.random.randn(1000)
plt.hist(data, bins=30, norm
color='steelblue', edgecolor='r
plt.show()
```



HISTOGRAMS

- Parameter 'alpha' → used to adjust transparency of the bars
- ****kwargs** → This syntax allows to pass additional keyword arguments to the hist function. These arguments can include parameters like bins, color, alpha, etc., which control the appearance and behavior of the histogram.

```
import matplotlib.pyplot as plt
import numpy as np
x1 = np.random.normal(0, 0.8, 1000)
x2 = np.random.normal(-2, 1, 1000)
x3 = np.random.normal(3, 2, 1000)
kwargs = dict(histtype='stepfilled', alpha=0.3, bins=40)
plt.hist(x1, **kwargs)
plt.hist(x2, **kwargs)
plt.hist(x3, **kwargs)
plt.show()
```



HISTOGRAMS

- `np.histogram()` → used if the number of points that would be in a given bin has to be counted
- Number of bins can be altered as per user's need

```
import numpy as np
data = np.random.randn(1000)
counts, bin_edges = np.histogram(data, bins=5)
print(counts)
```

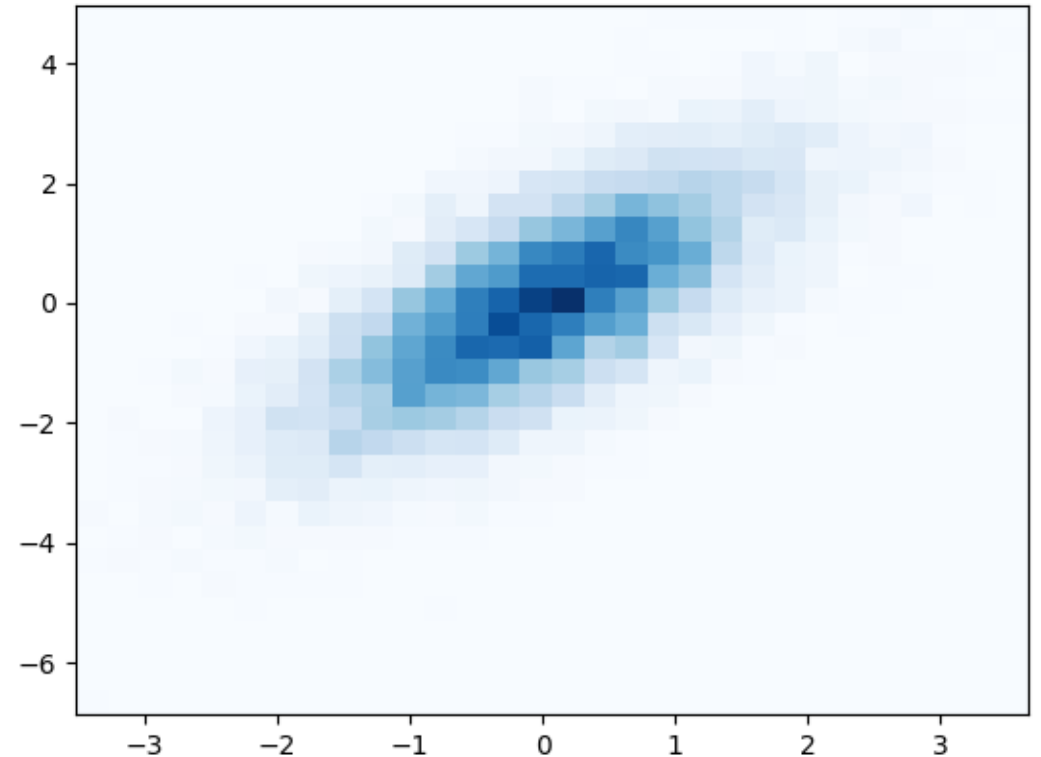
OUTPUT:

```
[ 77 374 435 106  8]
```

TWO-DIMENSIONAL HISTOGRAMS

- histograms in two dimensions can be created by dividing points among two-dimensional bins.
- Using `plt.hist2d()` function

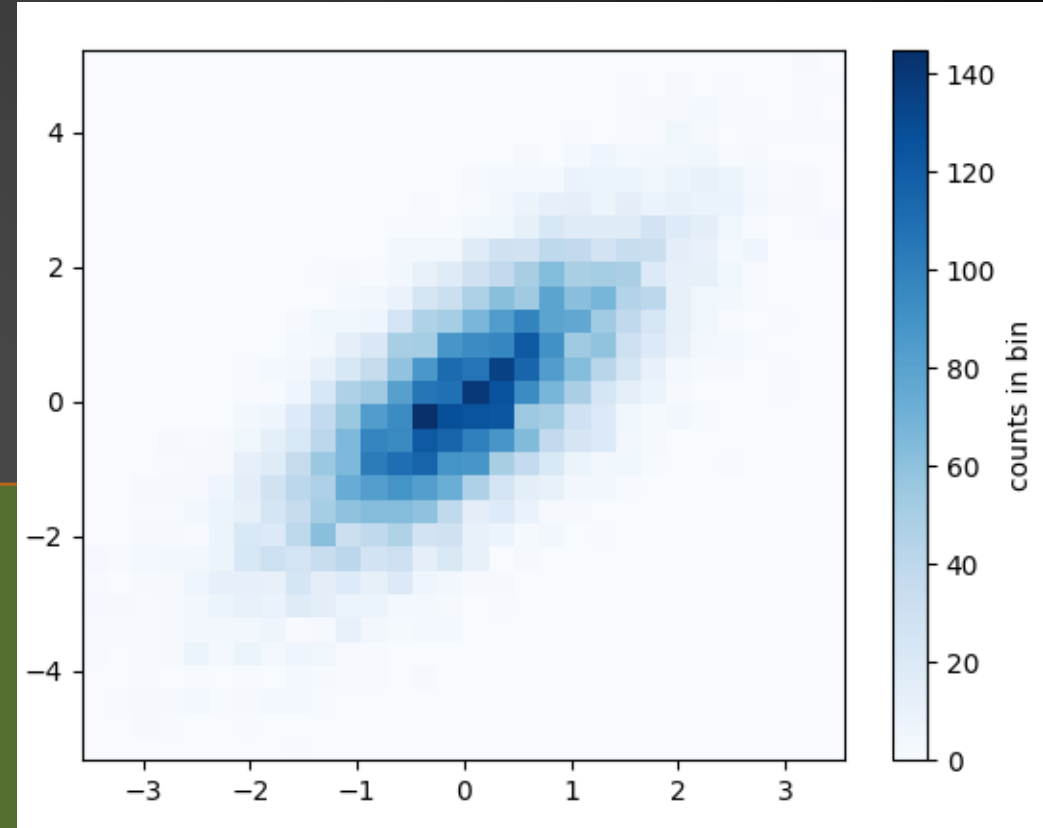
```
import matplotlib.pyplot as plt
import numpy as np
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 10000).T
plt.hist2d(x, y, bins=30, cmap='Blues')
plt.show()
```



TWO-DIMENSIONAL HISTOGRAMS

- Histograms in two dimensions can be created by dividing points among two-dimensional bins.
- Using `plt.hist2d()` function

```
import matplotlib.pyplot as plt
import numpy as np
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 10000).T
plt.hist2d(x, y, bins=30, cmap='Blues')
cb = plt.colorbar()
cb.set_label('counts in bin')
plt.show()
```



TWO-DIMENSIONAL HISTOGRAMS

- `np.histogram2d()` → used if the number of points in a given bin has to be counted
- Number of bins can be altered as per user's need

```
import numpy as np
data = np.random.randn(1000)
counts, xedges, yedges = np.histogram2d(x, y, bins=6)
print(counts)
```

- `xedges` and `yedges` – gives the bin edge points along the x-axis and y-axis

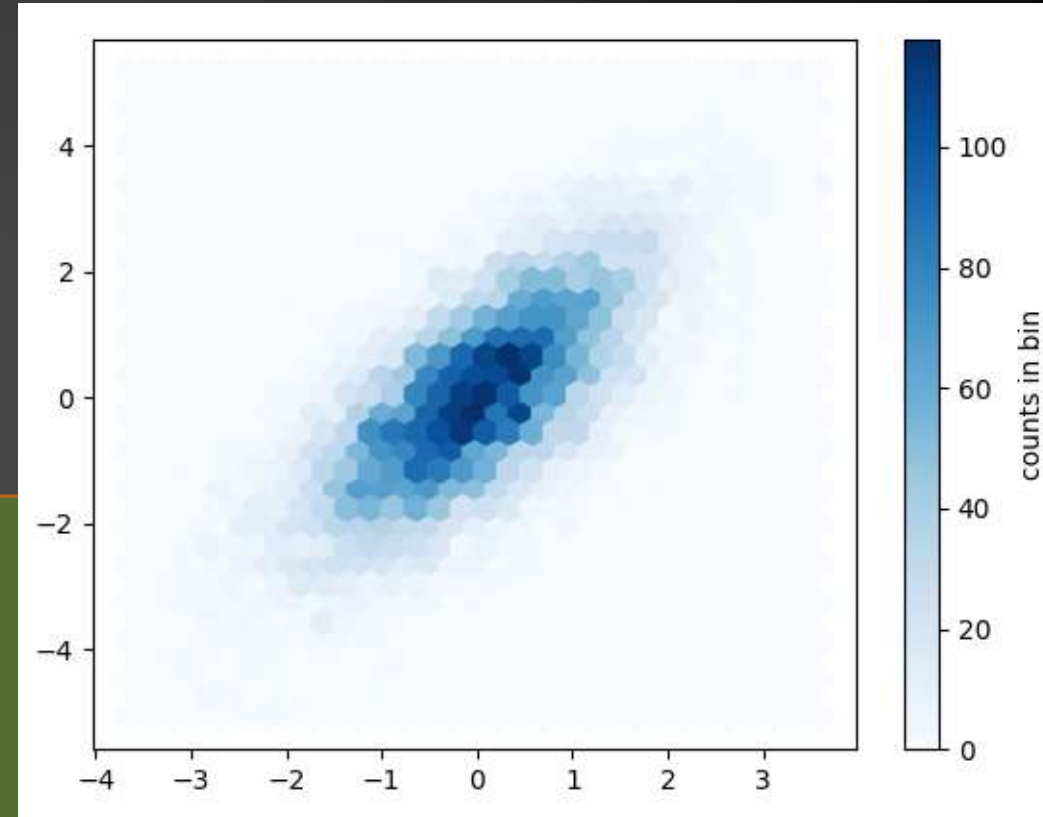
OUTPUT:

```
[[ 1.  2.  0.  0.  0.  0.]
 [ 1.  6.  2.  0.  0.  0.]
 [ 1.  9. 13.  8.  1.  0.]
 [ 0.  3.  7.  7.  8.  0.]
 [ 0.  0.  6.  9.  7.  1.]
 [ 0.  0.  0.  1.  2.  5.]]
```

TWO-DIMENSIONAL HISTOGRAMS

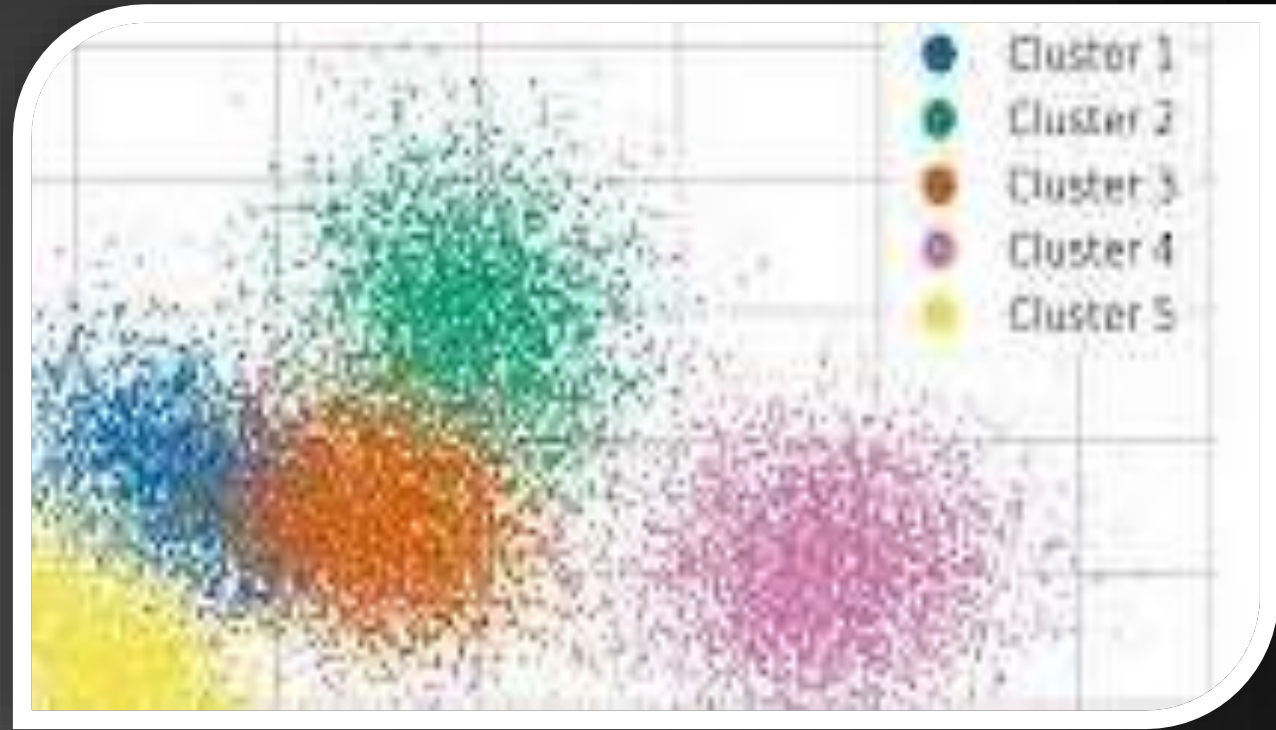
- Histograms in two dimensions can be created by a two-dimensional dataset binned within a grid of hexagons.
- Using `plt.hexbin()` function

```
import matplotlib.pyplot as plt
import numpy as np
mean = [0, 0]
cov = [[1, 1], [1, 2]]
x, y = np.random.multivariate_normal(mean, cov, 10000).T
plt.hexbin(x, y, gridsize=30, cmap='Blues')
cb = plt.colorbar(label = 'counts in bin')
plt.show()
```



PLOT LEGENDS

- **CUSTOMIZING PLOT LEGENDS**
- **CHOOSING ELEMENTS FOR THE LEGEND**
- **LEGEND FOR SIZE OF POINTS**
- **MULTIPLE LEGENDS**



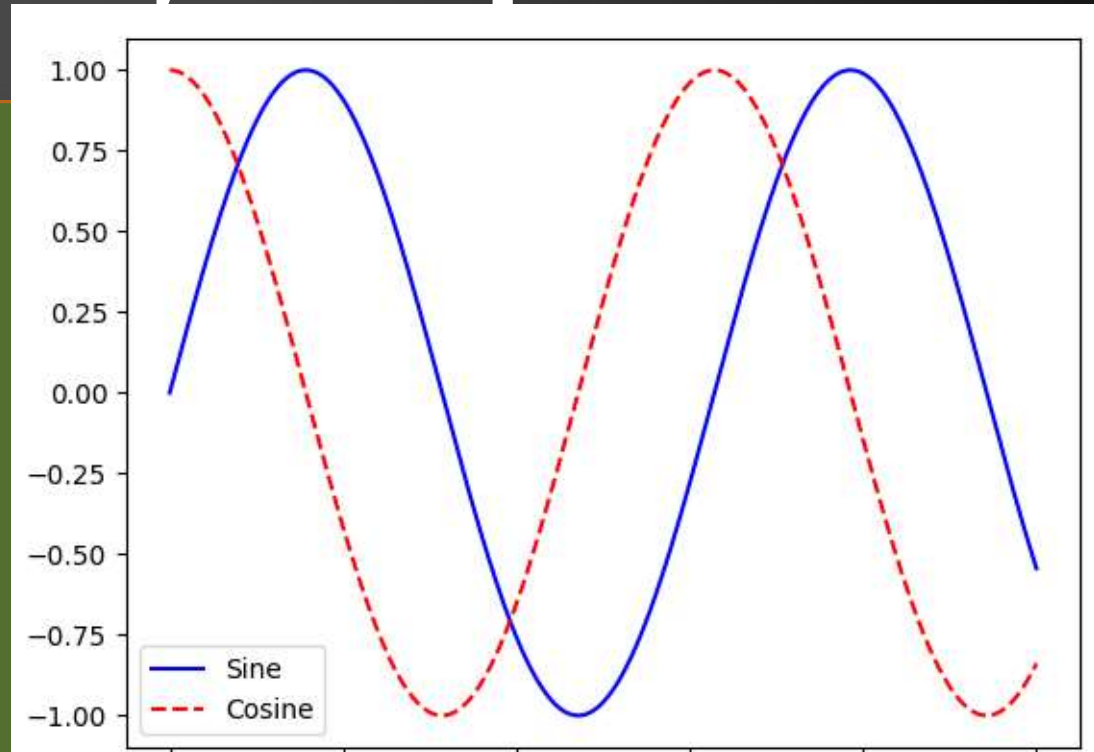
PLOT LEGENDS

- Plot legends → used to give meaning to a visualization by assigning labels to the various plot elements.

```
import matplotlib.pyplot as plt  
import numpy as np
```

- The simplest legend can be created with the `plt.legend()` command, which automatically creates a legend for any labeled plot elements

```
x = np.linspace(0, 10, 1000)  
plt.plot(x, np.sin(x), '-b', label='Sine')  
plt.plot(x, np.cos(x), '--r', label='Cosine')  
plt.legend()  
plt.show()
```



CUSTOMIZING PLOT LEGENDS

- A legend can be customized in location, border shape, etc.
 - specify the location → using 'loc' ('upper right', 'lower left',
 - turn off the frame → using 'frameon'
 - specify the number of columns in the legend → using 'ncol' command
 - specify a rounded box → using fancybox
 - add a shadow → using shadow
 - change the transparency (alpha value) of the frame → using framealpha
 - change the padding around the text → using borderpad

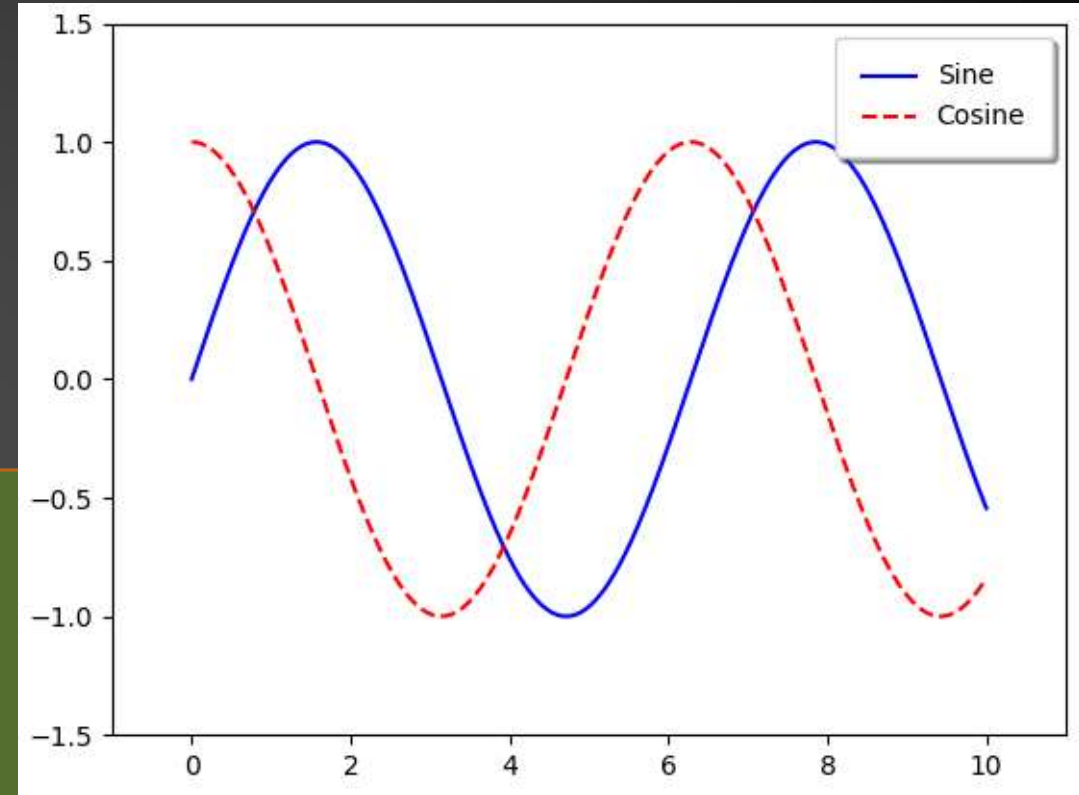
CUSTOMIZING PLOT LEGENDS

- Customizing a legend:

```
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x), '-b', label='Sine')
plt.plot(x, np.cos(x), '--r', label='Cosine')
plt.axis([-1, 11, -1.5, 1.5])
```

```
plt.legend(loc='upper right', fancybox=True, shadow=True,  
framealpha=1, borderpad=1)
```

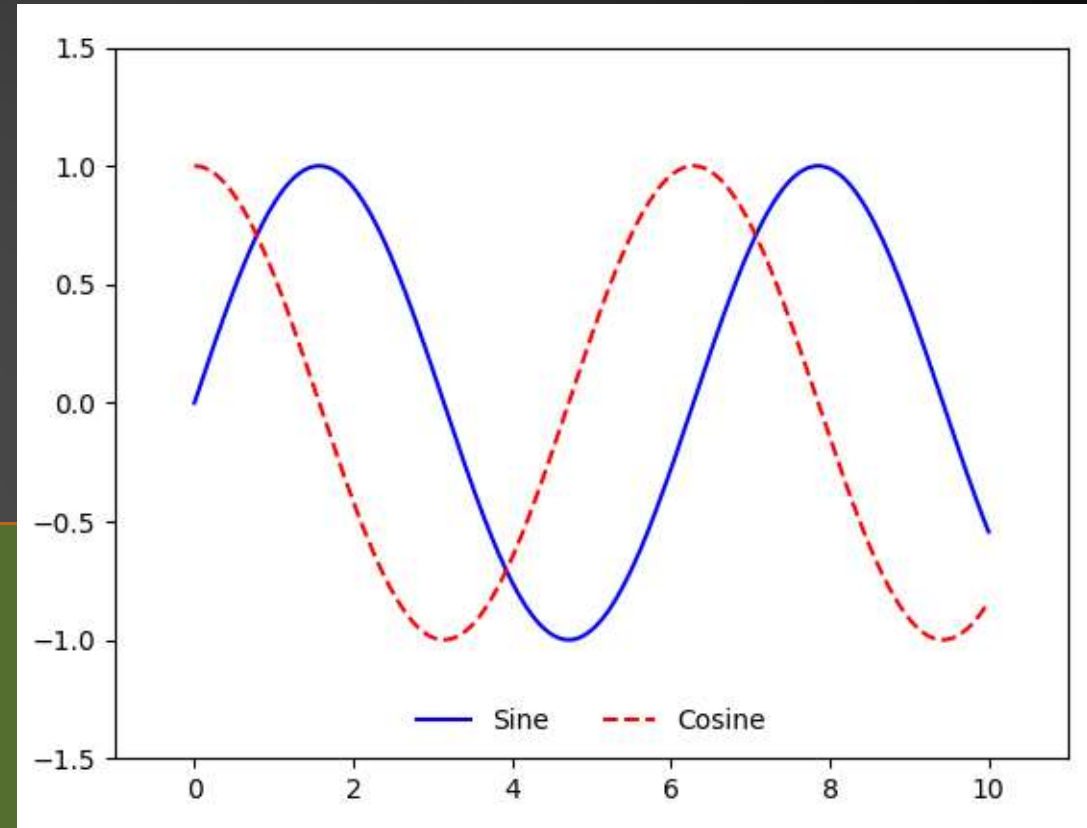
```
#plt.legend(frameon=False, loc='lower center', ncol=2)
plt.show()
```



CUSTOMIZING PLOT LEGENDS

- Customizing a legend:

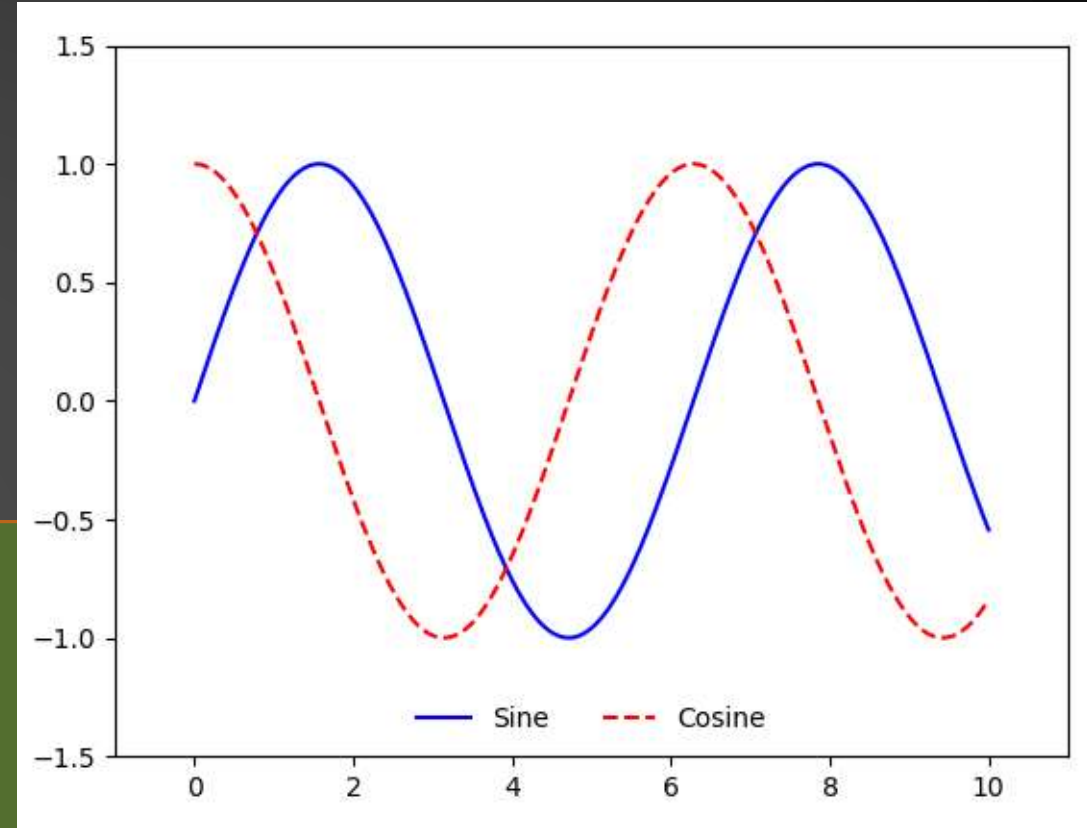
```
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x), '-b', label='Sine')
plt.plot(x, np.cos(x), '--r', label='Cosine')
plt.axis([-1, 11, -1.5, 1.5])
#plt.legend(loc='upper right', fancybox=True, shadow=True,
framealpha=1, borderpad=1)
plt.legend(frameon=False, loc='lower center', ncol=2)
plt.show()
```



CHOOSING ELEMENTS FOR THE LEGEND

- Customizing a legend:

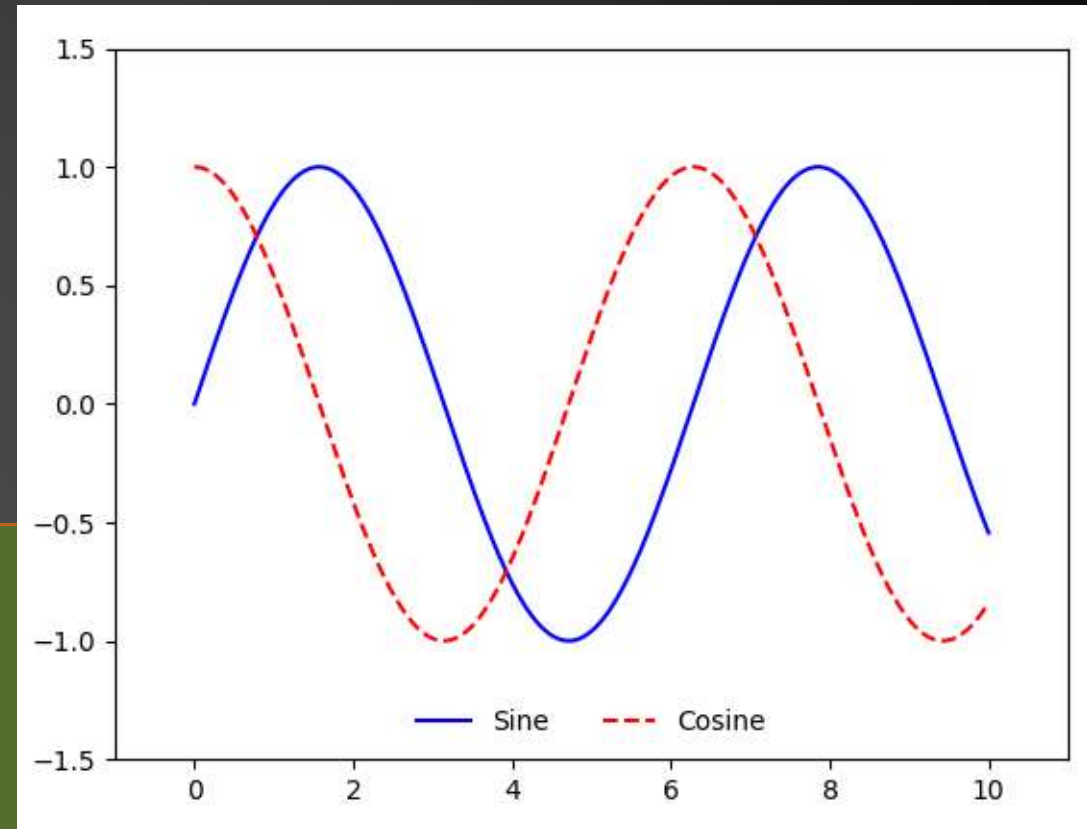
```
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x), '-b', label='Sine')
plt.plot(x, np.cos(x), '--r', label='Cosine')
plt.axis([-1, 11, -1.5, 1.5])
#plt.legend(loc='upper right', fancybox=True, shadow=True,
framealpha=1, borderpad=1)
plt.legend(frameon=False, loc='lower center', ncol=2)
plt.show()
```



LEGEND FOR SIZE OF POINTS

- Customizing a legend:

```
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x), '-b', label='Sine')
plt.plot(x, np.cos(x), '--r', label='Cosine')
plt.axis([-1, 11, -1.5, 1.5])
#plt.legend(loc='upper right', fancybox=True, shadow=True,
framealpha=1, borderpad=1)
plt.legend(frameon=False, loc='lower center', ncol=2)
plt.show()
```



MULTIPLE LEGENDS

- Customizing a legend:

```
x = np.linspace(0, 10, 1000)
plt.plot(x, np.sin(x), '-b', label='Sine')
plt.plot(x, np.cos(x), '--r', label='Cosine')
plt.axis([-1, 11, -1.5, 1.5])
#plt.legend(loc='upper right', fancybox=True, shadow=True,
framealpha=1, borderpad=1)
plt.legend(frameon=False, loc='lower center', ncol=2)
plt.show()
```

